Assignment 2

Recommended readings

- Lecture slides as starting literature
- MDN Links within slides for details, especially: alert, prompt, confirm, Math.random, Number(), DOM, checkValidity

Note: All exercises must be solved with plain JavaScript and CSS, not using additional libraries or frameworks.

Exercise 1 – JavaScript Basics, Scopes, Equality 0,1 Points

Take the quiz about JavaScript Concepts in Moodle. For this exercise, provide a screenshot of your achieved points in your submission.

Exercise 2 – JavaScript Basics: Number-Guessing Game I 0,1 Points

Disclaimer: This task is intended to cover all basic techniques in JavaScript and therefore does not adhere to best practices!

In this task, you will build a number-guessing game using JavaScript. Prepare a JS file that you embed in a basic HTML file with an empty body. You will work in this case with prompts and alerts for interaction with the user.

In the main part of your game, first create a constant for how many attempts a user shall have for guessing the right number (default: 3), and for storing the user's decision on whether to keep playing or not. Then create the game loop that continues as long as the user wants to play the game. Inside the loop, start by creating a variable `randomNumber` and assign it a generated random integer between 1 and 10 (inclusive) with the help of Math.random() and Math.floor(). Then, call the `playNumberGame` function passing the function the randomNumber. After each game, use `confirm()` to ask the user whether they want to play again. If not, display a *thank-you* message.

The function `playNumberGame` expects a `secretNumber` as parameter. The function shall further store all the user's guesses in an array. Next, create a loop to allow the user a certain number of attempts as stated by the before defined constant (default: 3). Inside the loop, call the `getUserInput` function to get the user's guess and store the return value in a variable called `guess`, and add it to the array. The next part checks the user input: If the `guess` is not a number, then alert the user with the variable, which now contains an error text, using `alert()`. Otherwise, if the number is equal to the `secretNumber`, alert the user with a message that the guess was correct, how many attempts were needed, and

return true. Otherwise, if the number was too high or low, alert the user about it, including how many attempts were already made. Then alert the user with a message containing the previous guesses, i.e., printing the array. If the user runs out of attempts, display an alert message revealing the correct number and return false.

The `getUserInput` function prompts the user to enter a number between 1 and 10 and stores the guess in a variable (`prompt()`). Convert the user's input to a number using Number(). Use a try...catch block to handle potential errors: If the input is not a number, or if it's less than 1 or greater than 10, throw a new Error with the message "Invalid input. Please enter a number between 1 and 10." If the input is valid, return the number. If an error occurs, catch it and return the error message.

Exercise 3 – DOM Operations 0,2 Points

---

1. Given the HTML code in ex3.html, draw a sketch of its DOM (see lecture slide 3-29).

2. Embed this JavaScript in your page. Open the page in your browser and explain each line of the output in detail:

```
var link = document.getElementsByTagName("a")[1];
console.log(link.attributes["href"].nodeValue);
console.log(link.getAttributeNode("href").nodeValue);
console.log(link.href);
console.log(link.firstChild);
console.log(link.attributes[0].nodeType);
console.log(link.firstChild.nodeType);
console.log(link.parentNode.nodeName);
console.log(link.lastChild.parentNode.nodeName);
console.log(link.attributes.length);
console.log(link.innerHTML);
console.log(link.outerHTML);
console.log(link.innerText);
console.log(link.outerText);
console.log(link.textContent);
console.log(link);
console.log(link.nodeValue);
```

3. Finally, explain the effect of this JavaScript code:

```
var link = document.getElementsByTagName("a").item(2);
link.removeAttribute("target");
```

Exercise 4 – Calculating Glass Price 0,2 Points

Given the glass order in the file *glassOrder.html*. Implement the following functionalities in the file calculate.js. **Do not change the HTML file.**

- Delete the paragraph with the duplicated heading.
- Add a read-only price input element to the glass order form with the label *price*.
- Implement a function *calculate(width,height,thickness)* that returns the price based on the formula:
    - price = width * height * thickness / 32 if the glass is colored or
    - price = width * height * thickness / 50 if the glass is in a shade of grey

      (Hint: R, G, and B channels have the same value for shades of grey)

  All inputs are assumed to be in cm. You may need to convert the values accordingly.
- Implement an *update()* function that calculates the price and sets the value of the price input field accordingly if all required inputs are given and have valid values. Use the *checkValidity()* function for checking. The input field should show a hyphen (-) if no valid values are given.
- Attach the update function as an event handler for the relevant input elements such that update() is called whenever a value is changed.

Exercise 5 – Event Listeners and Target 0,2 Points

Given the files in folder ex5, do the following steps:

1. Look at the code and think about its behavior. What would be the output in the console if you clicked on the button?
2. Now, run the script and check if the output in the console fits your assumption.
3. What is the difference between `target` and `currentTarget`? What is `this`?
4. Now, you want to ensure that only the information from the button is printed when clicked. Extend the code in ex5.js accordingly.

Don't change the provided ex5.html file; only *add* code to the ex5.js file!

Exercise 6 – Image Gallery 0,2 Points

Given the picture gallery from the lecture slides found in the folder *gallery*. First, familiarize yourself with the existing files: What is the HTML structure like? What does the code in gallery.js do?

Extend the gallery such that when a picture is clicked and shown in full window mode, the user has controls for:

- "*previous*", showing the previous picture. If not available, the last one.
- "*next*", showing the next image. If not available, the first one.

Important: Your script must work with any HTML content following the format of the HTML document of the lecture. **Only add new code to gallery.js! Don't change the existing code!**

Recommended Project Preparations

Consider everything you learned in this assignment for planning which input fields will need to be updated, added, or deleted via the DOM. You may also already be thinking about validity checks, as in Exercise 4.